

🍴 Zomato Restaurant Dataset — Exploratory Data Analysis

Alfido Tech Data Science Internship | Task 1

Objective: Analyze Zomato restaurant data to extract insights on ratings, cuisines, pricing, and location patterns.

Dataset: [Kaggle — Zomato Dataset by Bhanu Pratap Biswas](#)

📦 Step 1 — Install & Import Libraries

```
# Colab: install any missing packages
# !pip install wordcloud -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import warnings
import re

from collections import Counter

warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid', palette='muted', font_scale=1.1)
plt.rcParams.update({'figure.dpi': 120, 'axes.titleweight': 'bold'})

print('All libraries loaded successfully ✅')
```

All libraries loaded successfully ✅

📁 Step 2 — Load the Dataset

Upload `zomato.csv` to your Colab session or adjust the path below.

```
# ---- adjust path as needed ----
df = pd.read_csv('zomato.csv', encoding='latin-1')

print(f'Shape: {df.shape[0]:,} rows × {df.shape[1]} columns')
df.head(3)
```

Shape: 56,252 rows x 13 columns

	address	name	online_order	book_table	rate	votes	phone	location	rest_ty
0	942, 21st Main Road, 2nd Stage, Banashankari, ...	Jalsa	Yes	Yes	4.1/5	775	42297555\r\n+91 9743772233	Banashankari	Cas Dir
1	2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ...	Spice Elephant	Yes	No	4.1/5	787	080 41714161	Banashankari	Cas Dir
2	1112, Next to KIMS Medical College, 17th Cross...	San Churro Cafe	Yes	No	3.8/5	918	+91 9663487993	Banashankari	Cas Dir

Step 3 — Basic EDA: Data Types, Shape & Missing Values

```
print('=== Column Data Types ===')
print(df.dtypes)
print()
print('=== Missing Values ===')
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
missing_df = pd.DataFrame({'Missing Count': missing, 'Missing %': missing_pct})
print(missing_df[missing_df['Missing Count'] > 0])
```

```
=== Column Data Types ===
address          object
name             object
online_order     object
book_table       object
rate             object
votes            object
phone            object
location         object
rest_type        object
dish_liked       object
cuisines         object
approx_cost(for two people) object
listed_in(type)  object
dtype: object

=== Missing Values ===
```

	Missing Count	Missing %
address	17	0.03
name	16	0.03
online_order	19	0.03
book_table	58	0.10
rate	7838	13.93
votes	78	0.14
phone	1296	2.30
location	126	0.22
rest_type	338	0.60
dish_liked	28225	50.18
cuisines	203	0.36
approx_cost(for two people)	521	0.93
listed_in(type)	4610	8.20

```
=== Summary Statistics (Numeric Columns) ===
```

	address	name	online_order	book_table	rate	votes	phone	location	rest_type	dish_li
count	56235	56236	56233	56194	48414	56174	54956	56126	55914	28
unique	13397	11914	2639	2902	2877	5195	17712	2920	2961	8
top	('Rated 4.0'	('Rated 4.0'	Yes	No	NEW	0	('Rated 4.0'	BTM	Quick Bites	('Rated
freq	942	300	30444	45268	2208	10027	412	5125	19132	

✓ Step 4 — Data Cleaning

```
# --- 4a. Standardise column names ---
df.columns = [c.strip().lower().replace(' ', '_') for c in df.columns]
print('Columns after rename:', df.columns.tolist())
```

```
Columns after rename: ['address', 'name', 'online order', 'book table', 'rate', 'votes', 'phone', '']
```

```
# --- 4b. Clean the 'rate' column (e.g. '4.1/5', 'NEW', '-') ---
def clean_rate(val):
    if pd.isna(val):
        return np.nan
    val = str(val).strip()
    if val in ('NEW', '-', 'nan', ''):
        return np.nan
    # Extract numeric part before '/'
    match = re.search(r'(\d+\.\d*)', val.split('/')[0])
    return float(match.group(1)) if match else np.nan

df['rate_clean'] = df['rate'].apply(clean_rate)
print(f"Valid ratings: {df['rate_clean'].notna().sum():,} / {len(df):,}")
print(df['rate_clean'].describe())
```

```
Valid ratings: 43,447 / 56,252
count      43447.000000
mean         4.980667
std         64.393407
min          0.000000
25%          3.400000
50%          3.700000
75%          4.000000
max         8335.000000
Name: rate_clean, dtype: float64
```

```
# --- 4c. Clean 'approx_cost(for_two_people)' ---
cost_col = [c for c in df.columns if 'cost' in c][0]
df['cost_for_two'] = (
    df[cost_col]
    .astype(str)
    .str.replace(',', '', regex=False)
    .pipe(pd.to_numeric, errors='coerce')
)

print(f'Cost column: {cost_col}')
print(df['cost_for_two'].describe())
```

```
Cost column: approx_cost(for_two_people)
count    51371.000000
mean      555.431566
```

```
std      438.850728
min       40.000000
25%      300.000000
50%      400.000000
75%      650.000000
max     6000.000000
Name: cost_for_two, dtype: float64
```

```
# --- 4d. Boolean columns ---
if 'online_order' in df.columns:
    df['online_order'] = df['online_order'].map({'Yes': 1, 'No': 0})
if 'book_table' in df.columns:
    df['book_table'] = df['book_table'].map({'Yes': 1, 'No': 0})

# --- 4e. Drop duplicates (by restaurant name + location) ---
before = len(df)
name_col = [c for c in df.columns if 'name' in c][0]
loc_col = [c for c in df.columns if 'location' in c][0]
df.drop_duplicates(subset=[name_col, loc_col], inplace=True)
print(f'Removed {before - len(df):,} duplicate rows. Remaining: {len(df):,}')

Removed 40,291 duplicate rows. Remaining: 15,961
```

Step 5 — Visualisation 1: Distribution of Restaurant Ratings

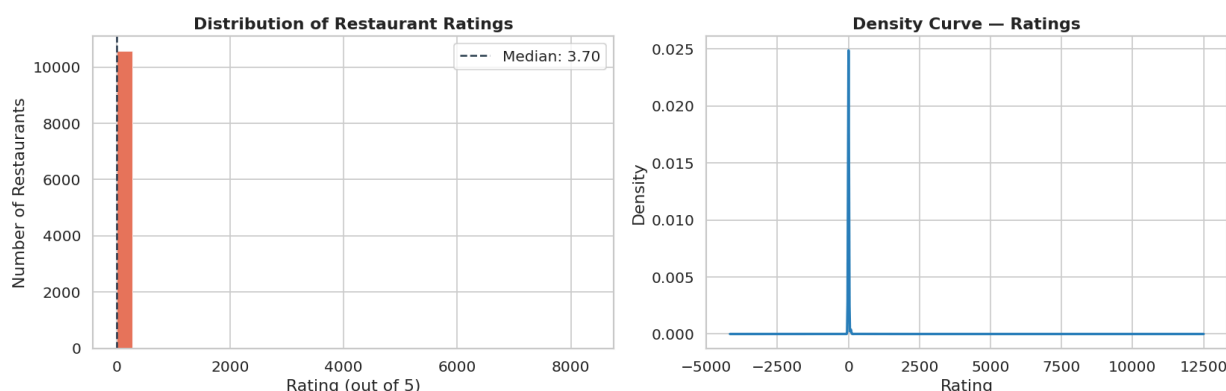
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogram
axes[0].hist(df['rate_clean'].dropna(), bins=30, color='#E8735A', edgecolor='white', linewidth=0.5)
axes[0].set_title('Distribution of Restaurant Ratings')
axes[0].set_xlabel('Rating (out of 5)')
axes[0].set_ylabel('Number of Restaurants')
median_rating = df['rate_clean'].median()
axes[0].axvline(median_rating, color='#2C3E50', linestyle='--', linewidth=1.5, label=f'Median: {median_rating}')
axes[0].legend()

# KDE
df['rate_clean'].dropna().plot.kde(ax=axes[1], color='#2980B9', linewidth=2)
axes[1].set_title('Density Curve – Ratings')
axes[1].set_xlabel('Rating')
axes[1].set_ylabel('Density')

plt.suptitle('Fig 1 – Rating Distribution', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('fig1_rating_distribution.png', bbox_inches='tight')
plt.show()
print('\nKey observation: Ratings are left-skewed; most restaurants cluster between 3.5 and 4.2.')
```

Fig 1 — Rating Distribution



Key observation: Ratings are left-skewed; most restaurants cluster between 3.5 and 4.2.

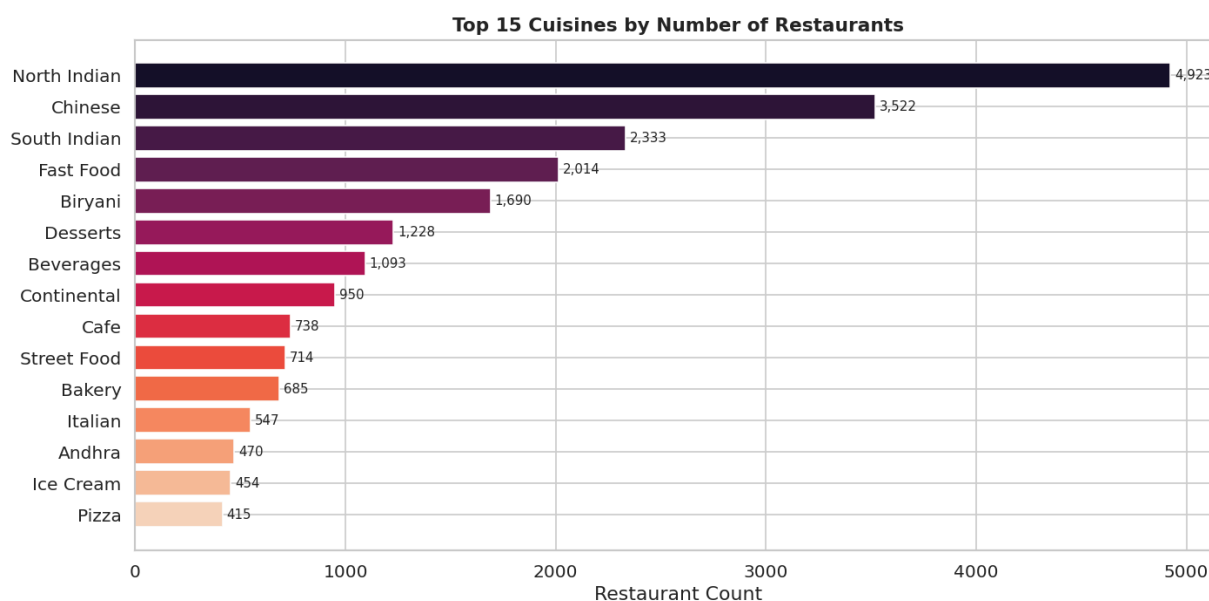
Step 6 — Visualisation 2: Top Cuisines by Restaurant Count

```
# Explode multi-cuisine entries (comma-separated)
cuisine_col = [c for c in df.columns if 'cuisines' in c][0]
all_cuisines = (
    df[cuisine_col]
    .dropna()
    .str.split(',')
    .explode()
    .str.strip()
)
top_cuisines = all_cuisines.value_counts().head(15)

fig, ax = plt.subplots(figsize=(12, 6))
colors = sns.color_palette('rocket', len(top_cuisines))
bars = ax.barh(top_cuisines.index[::-1], top_cuisines.values[::-1], color=colors[::-1])

for bar, val in zip(bars, top_cuisines.values[::-1]):
    ax.text(bar.get_width() + 20, bar.get_y() + bar.get_height()/2,
            f'{val:,}', va='center', fontsize=9)

ax.set_title('Top 15 Cuisines by Number of Restaurants', fontsize=13)
ax.set_xlabel('Restaurant Count')
ax.set_ylabel('')
plt.tight_layout()
plt.savefig('fig2_top_cuisines.png', bbox_inches='tight')
plt.show()
```



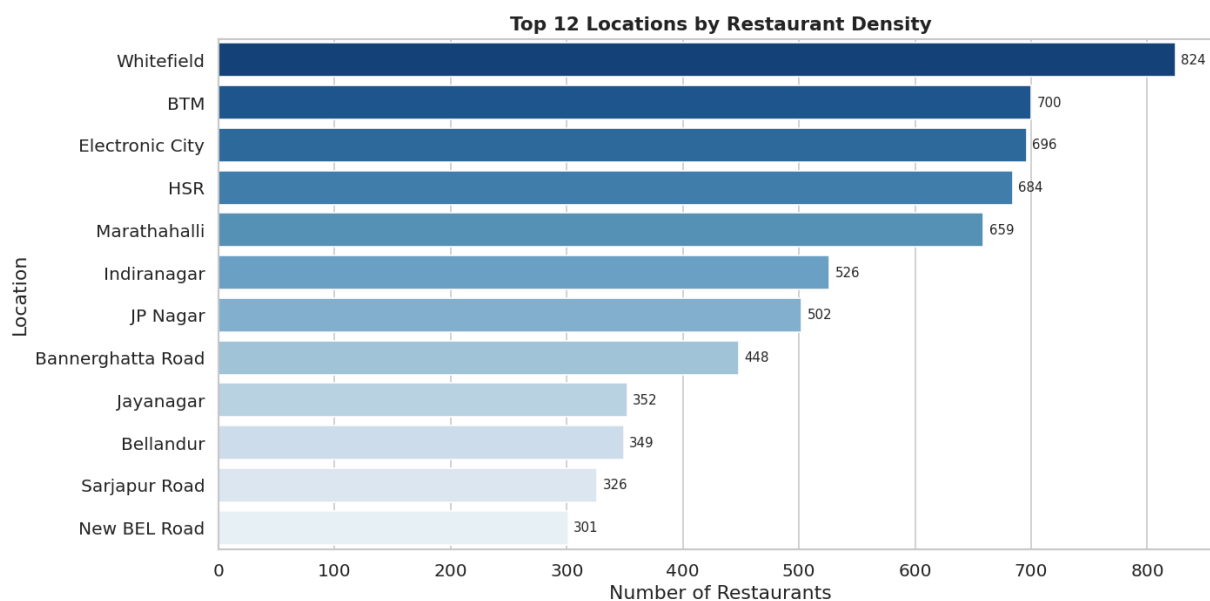
Step 7 — Visualisation 3: Location Hotspots

```
top_locs = df[loc_col].value_counts().head(12)

fig, ax = plt.subplots(figsize=(12, 6))
sns.barplot(x=top_locs.values, y=top_locs.index, palette='Blues_r', ax=ax)
ax.set_title('Top 12 Locations by Restaurant Density', fontsize=13)
ax.set_xlabel('Number of Restaurants')
ax.set_ylabel('Location')

for i, (val, loc) in enumerate(zip(top_locs.values, top_locs.index)):
    ax.text(val + 5, i, f'{val:,}', va='center', fontsize=9)
```

```
plt.tight_layout()
plt.savefig('fig3_location_hotspots.png', bbox_inches='tight')
plt.show()
```



Step 8 — Visualisation 4: Price vs Rating (Scatter + Binned Box Plot)

```
plot_df = df[['cost_for_two', 'rate_clean']].dropna()
plot_df = plot_df[plot_df['cost_for_two'] < plot_df['cost_for_two'].quantile(0.98)] # remove extreme values

# Create price buckets
plot_df['price_bucket'] = pd.cut(
    plot_df['cost_for_two'],
    bins=[0, 200, 400, 600, 800, 1000, 3000],
    labels=['₹0-200', '₹201-400', '₹401-600', '₹601-800', '₹801-1000', '₹1000+']
)

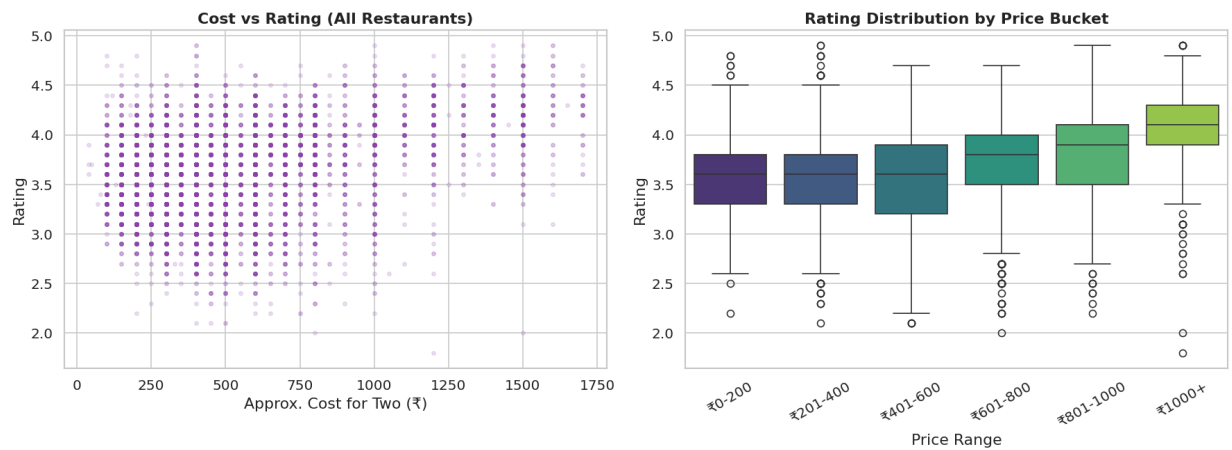
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Scatter
axes[0].scatter(plot_df['cost_for_two'], plot_df['rate_clean'],
                alpha=0.15, s=10, color='#8E44AD')
axes[0].set_xlabel('Approx. Cost for Two (₹)')
axes[0].set_ylabel('Rating')
axes[0].set_title('Cost vs Rating (All Restaurants)')

# Box plot by price bucket
sns.boxplot(data=plot_df, x='price_bucket', y='rate_clean',
            palette='viridis', ax=axes[1])
axes[1].set_title('Rating Distribution by Price Bucket')
axes[1].set_xlabel('Price Range')
axes[1].set_ylabel('Rating')
axes[1].tick_params(axis='x', rotation=30)

plt.suptitle('Fig 4 – Price vs Rating Analysis', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('fig4_price_vs_rating.png', bbox_inches='tight')
plt.show()
```

Fig 4 — Price vs Rating Analysis



Step 9 — Visualisation 5: Correlation Heatmap

```

votes_col = [c for c in df.columns if 'vote' in c]

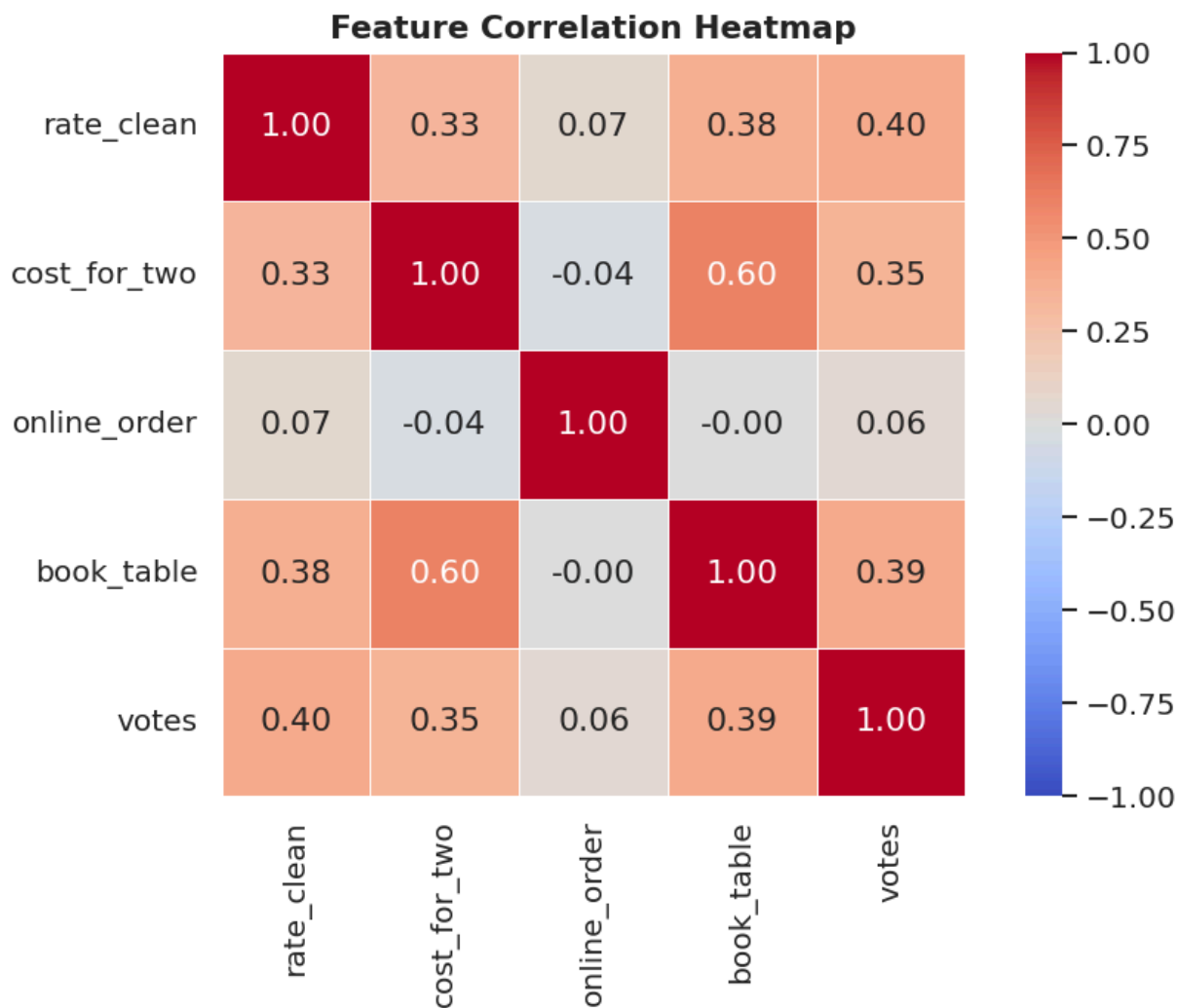
# Convert votes column to numeric (force non-numeric to NaN)
if votes_col:
    df[votes_col[0]] = pd.to_numeric(df[votes_col[0]], errors='coerce')

num_cols = ['rate_clean', 'cost_for_two']
if 'online_order' in df.columns: num_cols.append('online_order')
if 'book_table' in df.columns: num_cols.append('book_table')
if votes_col: num_cols.append(votes_col[0])

corr = df[num_cols].corr()

fig, ax = plt.subplots(figsize=(8, 6))
mask = np.triu(np.ones_like(corr, dtype=bool), k=1) # show lower triangle only
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, linewidths=0.5, ax=ax, mask=False,
            square=True, vmin=-1, vmax=1)
ax.set_title('Feature Correlation Heatmap', fontsize=13)
plt.tight_layout()
plt.savefig('fig5_heatmap.png', bbox_inches='tight')
plt.show()

```



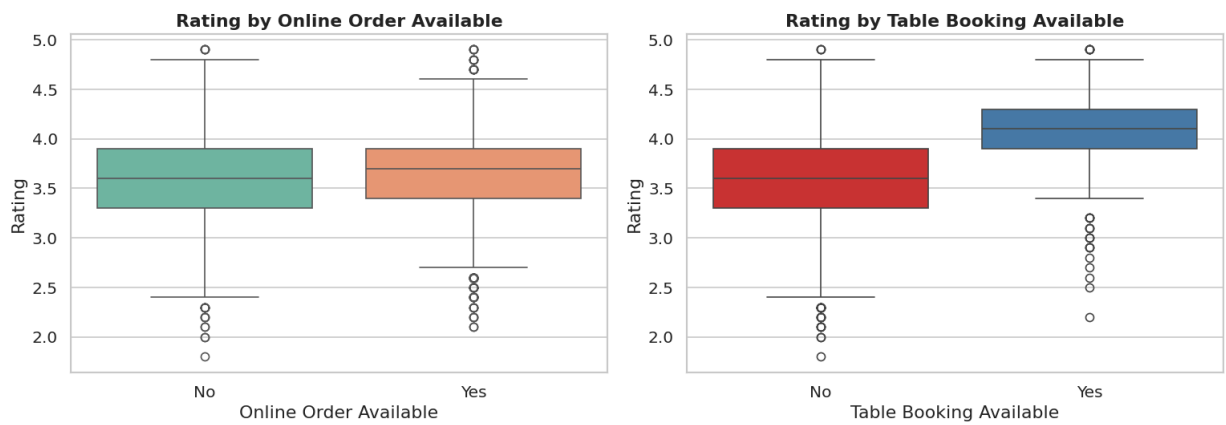
Step 10 — Visualisation 6: Online Order & Table Booking Impact

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for ax, col, label, pal in zip(
    axes,
    ['online_order', 'book_table'],
    ['Online Order Available', 'Table Booking Available'],
    ['Set2', 'Set1']
):
    if col not in df.columns:
        ax.text(0.5, 0.5, 'Column not found', ha='center', transform=ax.transAxes)
        continue
    sns.boxplot(data=df, x=col, y='rate_clean', palette=pal, ax=ax)
    ax.set_title(f'Rating by {label}')
    ax.set_xlabel(label)
    ax.set_ylabel('Rating')
    ax.set_xticks([0, 1])
    ax.set_xticklabels(['No', 'Yes'])

plt.suptitle('Fig 6 – Service Features vs Rating', fontsize=13, y=1.01)
plt.tight_layout()
plt.savefig('fig6_service_features.png', bbox_inches='tight')
plt.show()
```


Fig 6 — Service Features vs Rating



Step 11 — Word Cloud: Popular Cuisines

```
try:
    from wordcloud import WordCloud

    cuisine_text = ' '.join(all_cuisines.dropna().tolist())

    wc = WordCloud(
        width=900, height=450,
        background_color='white',
        colormap='RdYlGn',
        max_words=80,
        collocations=False
    ).generate(cuisine_text)

    fig, ax = plt.subplots(figsize=(13, 6))
    ax.imshow(wc, interpolation='bilinear')
    ax.axis('off')
    ax.set_title('Word Cloud - Popular Cuisines on Zomato', fontsize=14)
    plt.tight_layout()
    plt.savefig('fig7_wordcloud_cuisines.png', bbox_inches='tight')
    plt.show()
except ImportError:
    print('wordcloud not installed. Run: !pip install wordcloud')
```



```
if votes_col:
    vc = votes_col[0]
    df[vc] = pd.to_numeric(df[vc], errors='coerce')
    top_rest = (
        df[[name_col, 'rate_clean', vc, 'cost_for_two', loc_col]]
        .dropna(subset=['rate_clean', vc])
        .sort_values([vc, 'rate_clean'], ascending=[False, False])
        .head(10)
        .reset_index(drop=True)
    )
    top_rest.index += 1
    print('=== Top 10 Highly-Voted & Rated Restaurants ===')
    print(top_rest.to_string())
```

=== Top 10 Highly-Voted & Rated Restaurants ===

	name	rate_clean	votes	cost_for_two	location
1	Byg Brewski Brewing Company	4.9	16345.0	1600.0	Sanjapur Road
2	Toit	4.7	14956.0	1500.0	Indiranagar
3	Truffles	4.7	14654.0	900.0	Koramangala 5th Block
4	AB's - Absolute Barbecues	4.8	12121.0	1600.0	Marathahalli
5	The Black Pearl	4.7	10413.0	1400.0	Koramangala 5th Block
6	Onesta	4.4	9041.0	600.0	Koramangala 4th Block
7	Big Pitcher	4.7	9021.0	1800.0	Old Airport Road
8	Arbor Brewing Company	4.5	8375.0	2000.0	Brigade Road

9	Empire Restaurant	4.2	8176.0	750.0	Indiranagar
10	Prost Brew Pub	4.5	7854.0	1800.0	Koramangala 4th Block

Step 13 — Cuisine vs Average Rating

```
# Explode cuisine, join rating
expanded = (
    df[[cuisine_col, 'rate_clean']]
    .dropna()
    .assign(**{cuisine_col: df[cuisine_col].str.split(',')})
    .explode(cuisine_col)
    .assign(**{cuisine_col: lambda x: x[cuisine_col].str.strip()})
)

cuisine_rating = (
    expanded.groupby(cuisine_col)['rate_clean']
    .agg(['mean', 'count'])
    .query('count >= 50')          # at least 50 restaurants for reliability
    .sort_values('mean', ascending=False)
    .head(15)
)

fig, ax = plt.subplots(figsize=(12, 6))
colors = sns.color_palette('YlGn', len(cuisine_rating))
bars = ax.barh(cuisine_rating.index[::-1], cuisine_rating['mean'].values[::-1], color=colors)
for bar, val in zip(bars, cuisine_rating['mean'].values[::-1]):
    ax.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height()/2,
            f'{val:.2f}', va='center', fontsize=9)
ax.set_xlim(3, 5)
ax.set_title('Top 15 Cuisines by Average Rating (min 50 restaurants)', fontsize=12)
ax.set_xlabel('Average Rating')
plt.tight_layout()
plt.savefig('fig8_cuisine_avg_rating.png', bbox_inches='tight')
plt.show()
```



Step 14 — Summary & Key Findings

```
print('=' * 65)
print('                ZOMATO EDA — KEY FINDINGS SUMMARY')
print('=' * 65)

print(f"""
Dataset Overview
    • Total restaurants (post-dedup) : {len(df):,}
    • Unique locations                : {df[loc_col].nunique()}
    • Unique cuisine types            : {all_cuisines.nunique()}

Ratings
    • Mean rating    : {df['rate_clean'].mean():.2f}
    • Median rating  : {df['rate_clean'].median():.2f}
    • Restaurants without rating : {df['rate_clean'].isna().sum():,}

Pricing
    • Median cost for two : ₹{df['cost_for_two'].median():.0f}
    • Mean cost for two   : ₹{df['cost_for_two'].mean():.0f}

Service Features
    • Online order available : {df['online_order'].sum():,} ({df['online_order'].mean()*100:.1f}%
    • Table booking available: {df['book_table'].sum():,} ({df['book_table'].mean()*100:.1f}%)

Top Cuisine    : {top_cuisines.index[0]} ({top_cuisines.iloc[0]:,} restaurants)
```

```
Top Location : {df[loc_col].value_counts().index[0]} ({df[loc_col].value_counts().iloc[0],} res
""")

print('=' * 65)
print('          5 RECOMMENDATIONS FOR ALFIDO TECH')
print('=' * 65)
print("""
1. PARTNERSHIP FOCUS – Target top-rated, high-vote restaurants in
   dense locations (Koramangala, BTM, Indiranagar) for sponsored
   content & affiliate deals.

2. CONTENT IDEA – 'Cuisine Explorer' series: spotlight cuisines
   with high avg ratings but low representation (hidden gems).

3. ONLINE ORDER TREND – Restaurants with online ordering score
   higher on average; push vendors to enable digital ordering.

4. PRICE TIERING – Introduce a 'Value Picks' tag for highly-rated
   restaurants below ₹400 for two – drives engagement from
   budget conscious users
```